

Srivisweswara
Mohan Santhi

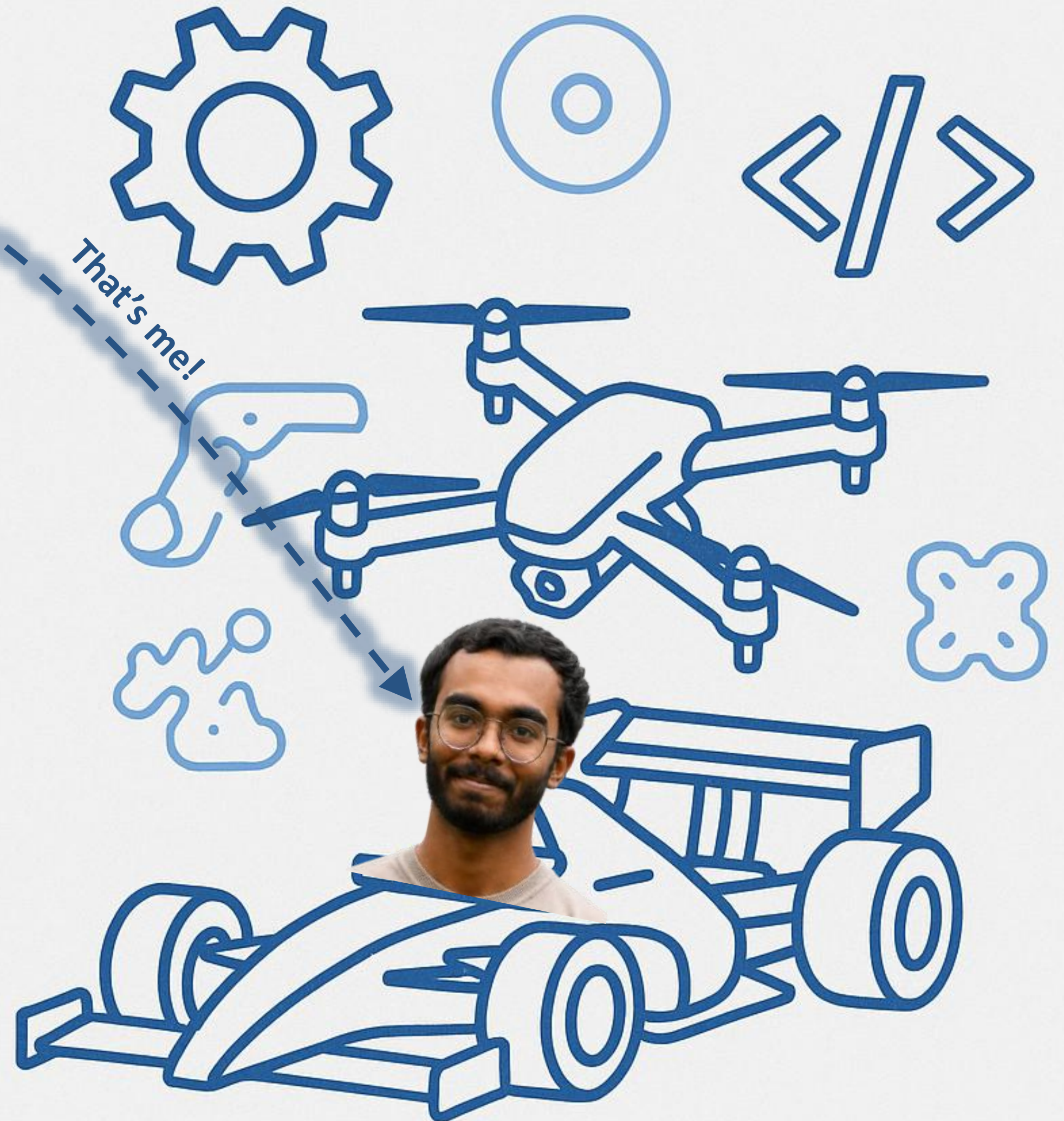
PORTFOLIO OF PROJECTS AND ACHIEVEMENTS

Engineer | Innovator | Developer

vichu26092003@gmail.com

<https://www.linkedin.com/in/srivisweswara-mohan-santhi-2a2461233/>

<https://srivisweswara.com/>



Overview

- **Purpose of this deck:**

I am highly passionate about Electric Vehicles and Embedded Systems. During my studies, I have been a technical lead for several teams that have won national level competitions and received acclaim from various organizations in India.

This deck serves to highlight my technical accomplishments.

- **About Me**

2026 MSc Embedded Systems Engineering from University of Leeds.

2025 B.E. Electrical and Electronics Engineering from Sri Krishna College of Engineering and Technology.

Work mainly with ARM-based processors and focus on building and programming hardware.

Proficient in Embedded C, C++, Rust (Intermediate), Assembly (Intermediate) and Data Structures & Algorithms.

Hands-on knowledge drone and Electric Vehicle building, PID tuning.

Lead technical projects and working with teams to turn ideas into real, working systems.

2024 & 2025 National level EV competition (EKVC) Champions

2023 SIH winner for national level hackathon for design and build of a drone

- **Projects highlighted in this slide deck**

Embedded Systems Projects

Electric Go-Kart

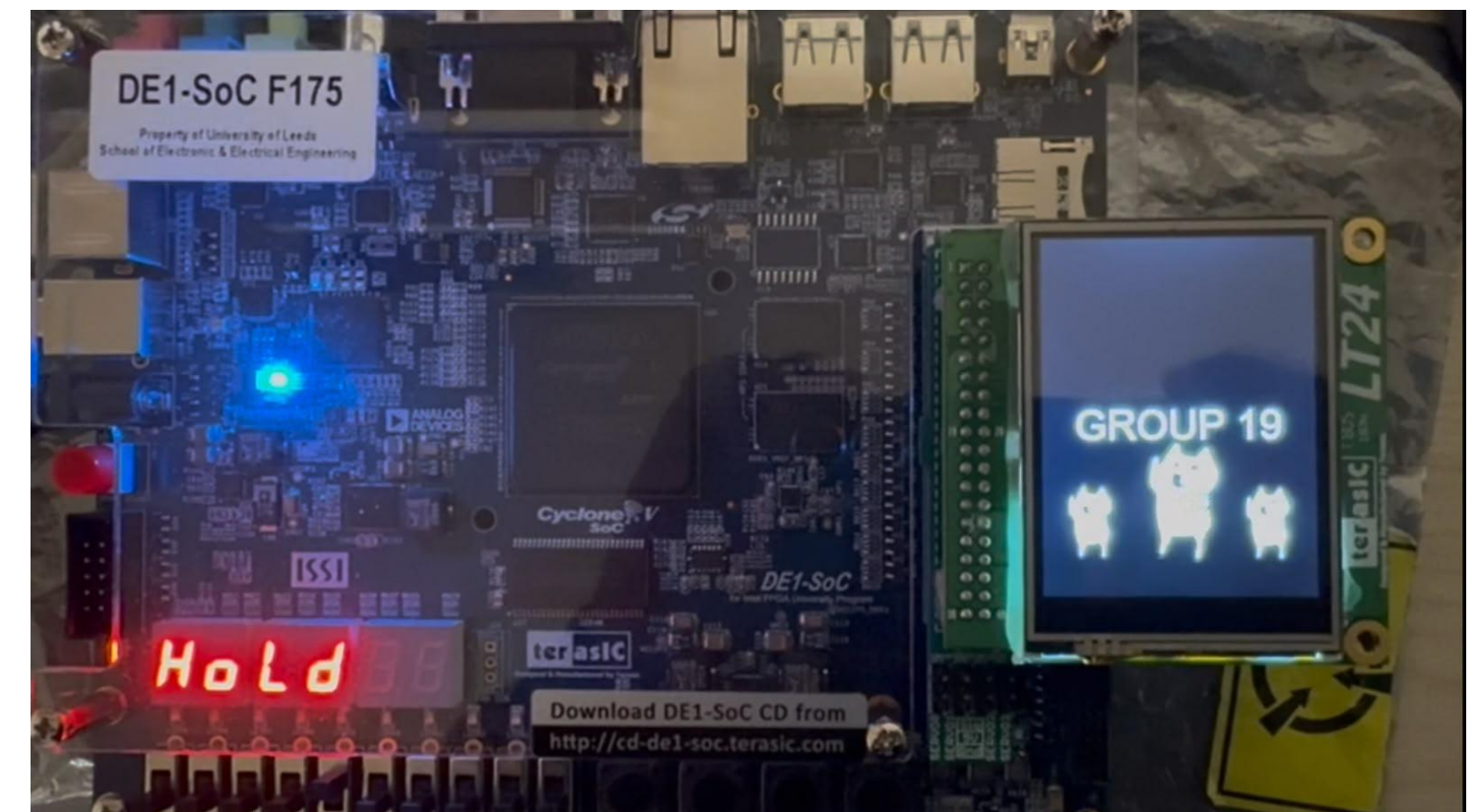
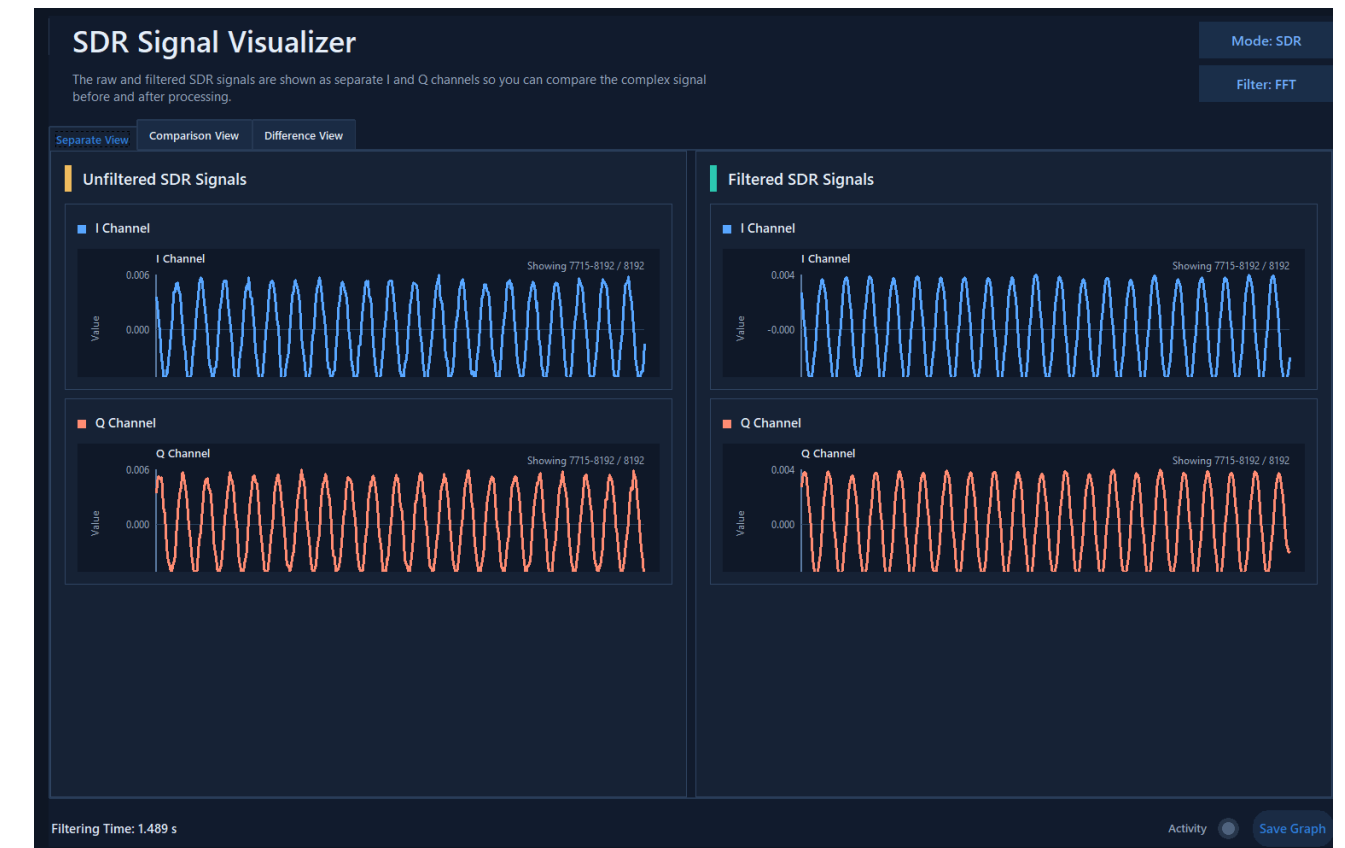
Drone

Websites



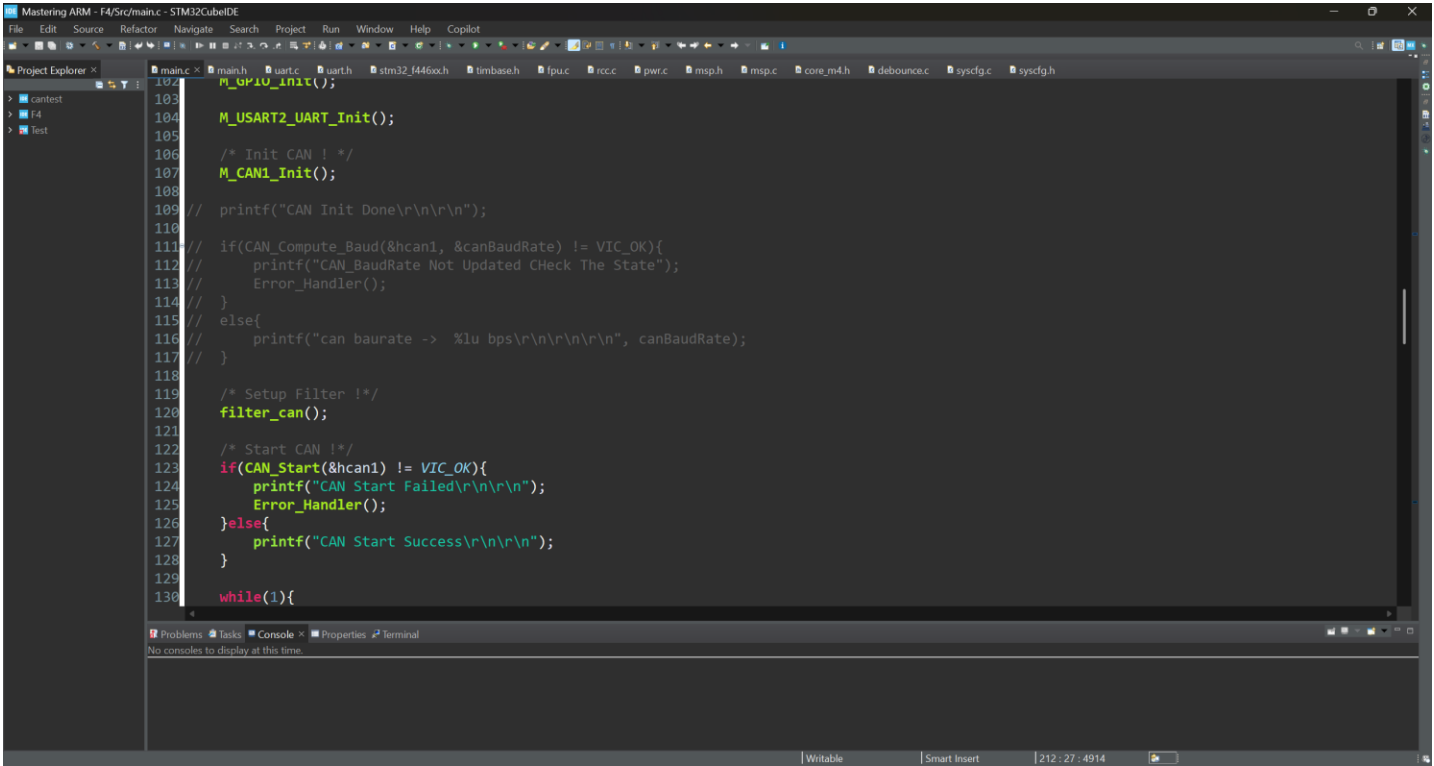
Cortex-A9 Bare-Metal DSP System

- **Project Focus**
- **Platform:** DE1-SoC, Cortex-A9, Cyclone V, audio and display peripherals
- **What I learned:** system-level bring-up, DSP optimisation, and getting several low-level blocks to work together cleanly
- **Why it matters:** I can help a company debug hard embedded problems, tune bottlenecks, and integrate hardware into a reliable product
- **Overview**
- Built a bare-metal DSP platform on the DE1-SoC that runs audio, SDR, sensor, and image-processing pipelines without an operating system.
- **Highlights**
- **Performance:** Used ARM NEON to reduce FIR and FFT runtimes with clear, measured gains.
- **Integration:** Brought UART, codec, LCD, storage, and board controls into one working runtime.
- **Company value:** This is the kind of work I can bring when a team needs low-level ownership, profiling, and dependable embedded integration.
- **Links**
- [Project Page](#) | [GitHub](#)



Bare-Metal STM32F4 Library

- Project Focus
- **Core:** STM32F4, startup code, linker scripts, and reusable low-level drivers
- **What I learned:** how to control boot flow, memory layout, and peripheral behaviour instead of treating them as black boxes
- **Why it matters:** I can step into low-level firmware work, debug with confidence, and build foundations that other engineers can extend
- Overview
- Built a from-scratch STM32F4 firmware base to own the startup flow, memory map, and register-level drivers.
- Highlights
- **Startup:** Worked directly with linker scripts, reset flow, and hardware initialisation.
- **Drivers:** Kept the code close to the registers so behaviour stays predictable during debugging.
- **Company value:** Useful for products that need solid board bring-up, reusable BSP work, and firmware that is not dependent on generated code.
- Links
- [Project Page](#) | [GitHub](#)



The screenshot shows an IDE window titled "Mastering ARM - F4/5r/main.c - STM32CubeIDE". The code is in C and defines initialization functions for the STM32F4. The functions include:

- `M_GPIO_Init();`
- `M_USART2_UART_Init();`
- `/* Init CAN ! */`
- `M_CAN1_Init();`
- `/* Setup Filter ! */`
- `filter_can();`
- `/* Start CAN ! */`
- `if(CAN_Start(&hcan1) != VIC_OK){`
- `printf("CAN Start Failed\r\n\r\n");`
- `Error_Handler();`
- `}else{`
- `printf("CAN Start Success\r\n\r\n");`
- `}`
- `while(1){`

The code is written in a dark-themed editor with syntax highlighting. The bottom of the window shows a "Problems" tab and a "Console" tab, both of which are empty.

Encryption and Cryptography Library

- **Project Focus**

- **Core:** encryption workflows, reusable interfaces, and secure data handling

- **What I learned:** security work needs the same discipline as firmware work - clear interfaces, traceable behaviour, and careful data movement

- **Why it matters:** I can bring a more security-aware mindset to embedded and low-level software teams

- **Overview**

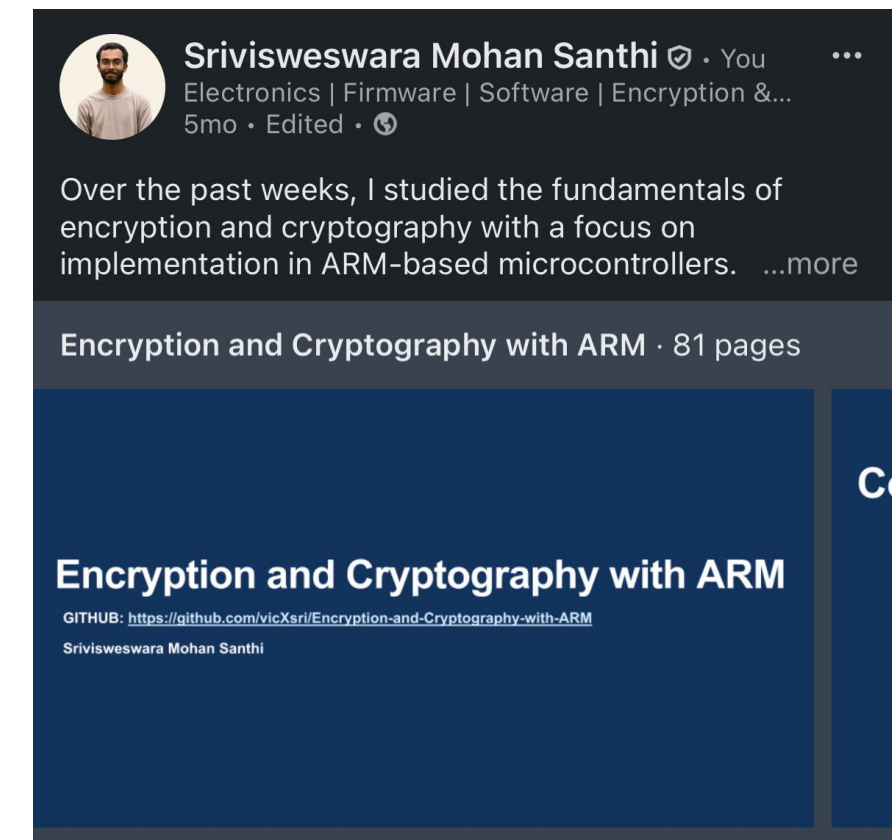
- Built this library to understand how encryption code should be structured, reused, and tested rather than treated as a one-off demo.

- **Highlights**

- **Design:** Focused on clean APIs and predictable handling of protected data.

- **Engineering:** Used the project to strengthen algorithm-level thinking beyond firmware alone.

- **Company value:** Useful for teams building products where correctness, trust, and clean software structure matter.



Links: [Project Page](#) | [GitHub Profile](#)

```
main.c aesDecrypt.c uart.c
301
302 void AES192_CFB(){}
320
321 void AES256_CFB(){}
340
341 void AES128_OFB(){}
359
360 void AES192_OFB(){}
378
379 void AES256_OFB(){}
398
399 void AES128_CTR(){}
417
418 void AES192_CTR(){}
436
437 void AES256_CTR(){}
438 /*AES256*/
439
440 if(AES256_Encrypt(AES_CTR_ENC, fulltext, sizeof(fulltext)
441 else printf("\nAES256 CTR Cipher Encrypt Success\n");
442
443 // if(AES256_Decrypt(AES_CTR_DEC, EncryptData256, sizeof(
444 // else printf("\nAES256 CTR Cipher Decrypt Success\n");
445
446 printf("\n\nEncrypted Data\n\n\n");
447 outputprint(EncryptData256Size, EncryptData256);
448
449 // printf("\n\nDecrypted Data\n\n\n");
450 // outputprint(DecryptData256Size, DecryptData256);
451
452 printf("\n\nEncrypt Size -> %d\n", EncryptData256Size);
453 // printf("\n\nDecrypt size -> %d\n", DecryptData256Size);
454
455 }
456
457 void outputprint(size_t length, uint8_t *data) {}
471
```

```
COMS - Tera Term VT
File Edit Setup Control Window Help
AddRoundKey Success
Round - 12
SubBytes Success
ShiftRows Success
MixColumns Success
AddRoundKey Success
Round - 13
SubBytes Success
ShiftRows Success
MixColumns Success
AddRoundKey Success
Round - 14
SubBytes Success
ShiftRows Success
AddRoundKey Success
AES256 CTR Cipher Encrypt Success
Encrypted Data
0x60 0x1E 0xC3 0x13 0x77 0x57 0x89 0x05 0xB7 0x07 0xF5 0x04 0xB8 0xF3 0x02 0x28
0xF4 0x43 0xE3 0xC0 0x40 0x62 0xB5 0x01 0xC0 0x84 0xE9 0x30 0xC0 0xC0 0xF5 0x65
0x28 0x09 0x30 0x00 0x02 0x30 0xF9 0x4C 0xF3 0x70 0x17 0x00 0x20 0x84 0x38 0x60
0x0F 0xC9 0xC5 0x80 0xB6 0x70 0x00 0xB6 0x13 0xC2 0x00 0xB8 0x45 0x79 0x41 0xB6
Encrypt Size -> 64
```


Retro Gaming Console

- Current Build

- **Platform:** STM32H7 handheld console build with LTDC display and custom firmware

- **What I learned:** staged bring-up, UI integration, and steady hardware debugging on a live build

- **Why it matters:** I can help teams move an early prototype from messy hardware tests to a usable product path

- Overview

- This project is teaching me to build the firmware stack step by step, from display and I2C bring-up to LVGL-based UI work.

- Highlights

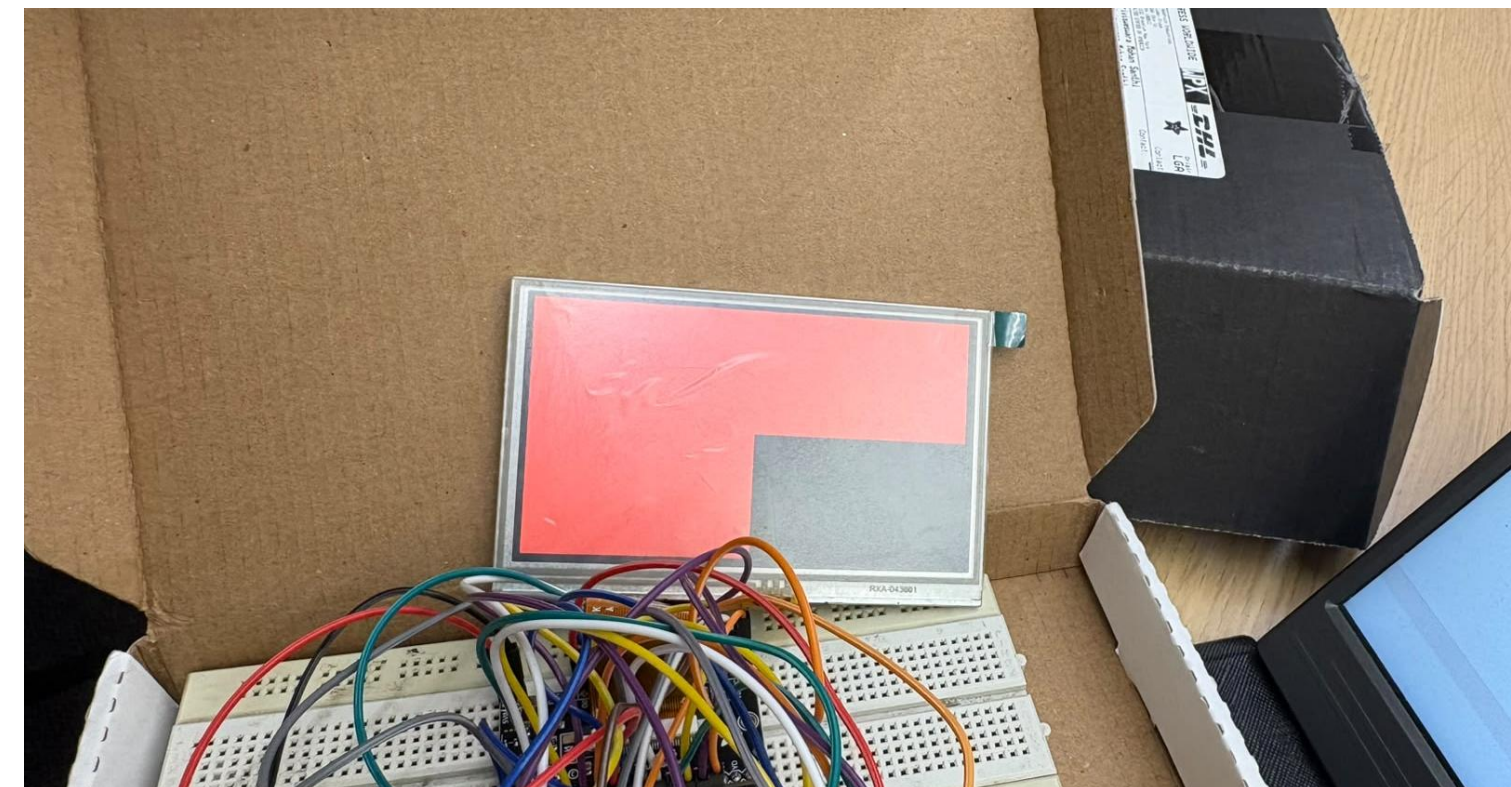
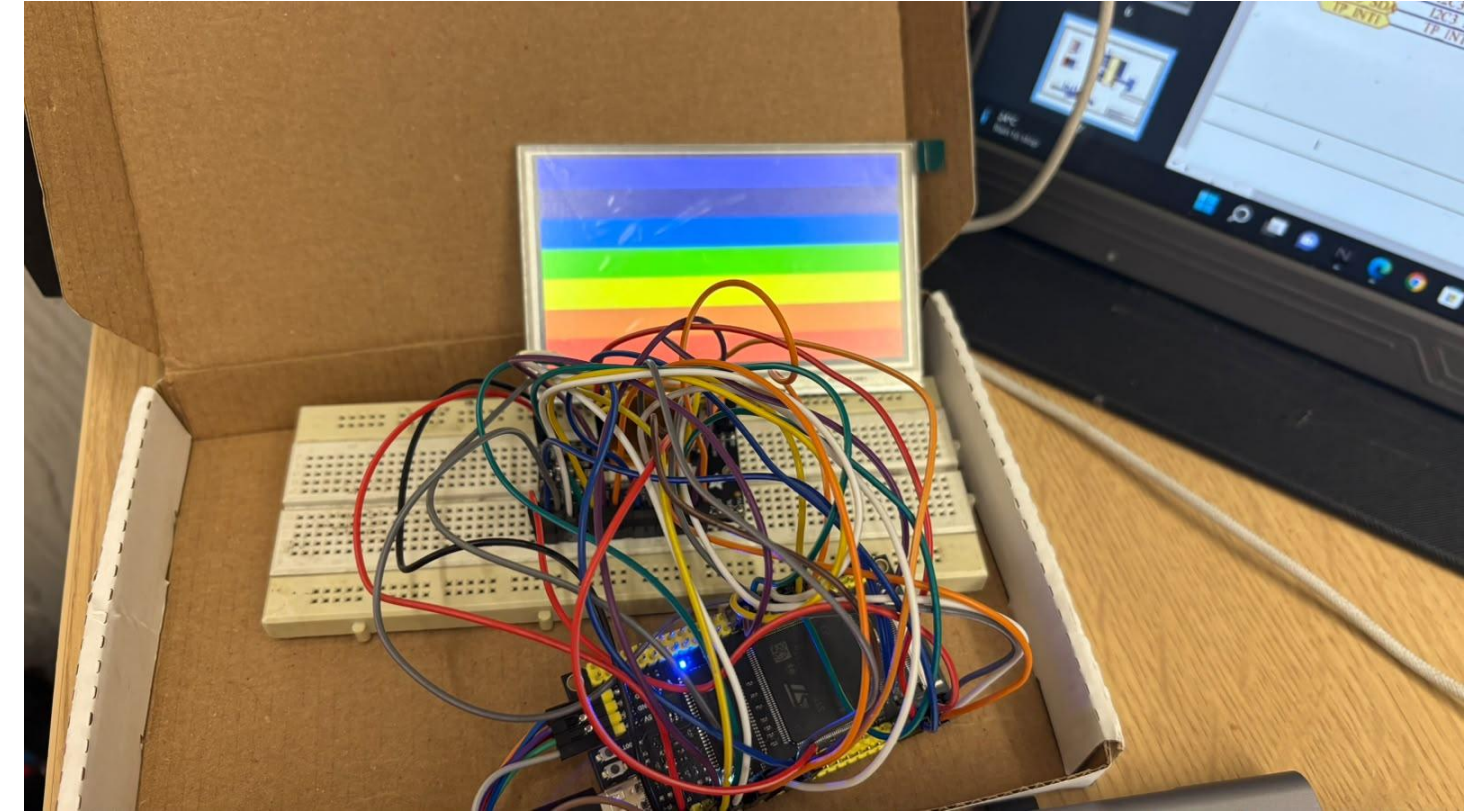
- **Bring-up:** Worked through LTDC, RCC, I2C, and display testing as the hardware came together.

- **UI Work:** Integrated LVGL and started rendering live data on the screen.

- **Company value:** Useful for products that need patient debugging, hardware-software coordination, and clear technical documentation.

- Links

- [Build Journal](#)



Electric Go-Kart – Gandiva Racing

Links: [Project](#) | [GitHub](#)

- **2022-2023: Chief Electrical Officer and Innovation Lead**
 - **Summary:** 1st season. Failed technical inspection. Finished last. Lots of learning.
 - Founding member of team to participate in national level EV competition to build 6 kW electric go-kart with 60V 150Ah NMC battery
 - Introduced F1 style steering wheel with custom instrument cluster
- **2023-2024: Chief Electrical Officer and Innovation Lead**
 - **Summary:** iterative improvements. Budget constrained. Solid technical progress.
 - Improved harness, reduced weight, reached 80km/hr.
 - Won EKVC 2024 championship
- **2024-2025: Promoted to Vice captain**
 - **Summary:** Our crowning achievement. My technical masterpiece 😊
 - Simplify complexity, Reduced weight to <120kg, Aerodynamic design, 104km/hr, custom 72V 40Ah HV battery pack with Molicel cells.
 - Won EKVC 2025 championship and placed All-India #3 at GKDC 2025

My specific contributions and technical details:

- **Electrical:**
 - Designed complete EV schematics in ACAD electrical. Designed and built a single integrated wiring harness
 - Used 25mm² silicone cables for high-current (up to 125 A at 72 V, peak 8.5 kW). 0.5 mm² for control lines, 0.25 mm² for signal-level electronics
 - Used 90-series waterproof connectors and automotive-grade insulation tape
- **Control Systems and PCB design:**
 - **PCB Design:** Altium Designer. Created a custom steering wheel PCB for control and feedback
 - **Microcontroller:** STM32F446RE. Developed embedded firmware using STM32CubeIDE
 - **Protocols Used:**
 - **CAN 2.0B:** For inter-module communication (BMS, display, logger)
 - Wrote firmware to manage request/response format (BMS) and continuous broadcast (MCU)
 - Used TJA1050 CAN transceiver
 - **SPI:** For WS182b LED strip control and SD card data logging
 - **UART:** For communication with the DWIN TFT display
- **Firmware**
 - Managed CAN message optimization to multiplex BMS + MCU data
 - Programmed speed-responsive LED animations for the WS182b strip
 - Wrote SPI and UART drivers for modular communication
 - Designed GUI and boot animation using DWIN software tools
- **Battery Pack Engineering**
 - Battery Configuration: 72V, 40Ah (Molicel cells)
 - Designed the pack architecture and validated sizing with MATLAB
 - Protection Devices:
 - 125 A MCCB (C-Rated) for HV safety
 - 250 A contactor for ignition and high-voltage line control
 - Simulation & Validation
- **MATLAB Simulink:**
 - Ran simulations using the Gandiva Drive Cycle to determine Wh/km and total Ah requirement
- **SolidWorks Flow Simulation:**
 - Validated aerodynamic CAD model. Verified airflow around body panels and cooling efficiency for motor and battery
 - Informed drag reduction and optimal component placement

V1 – 2022-2023



V2 – 2023-2024



V3 – 2024-2025



Electric Go-Kart – Gandiva Racing – Project details

- **LED light strip:** Designed an F1 style light strip on top of the steering wheel

Problem statement: During races, kart drivers could not easily read instrument cluster, leading to poor speed awareness.

Solution: I am very proud that this is my personal innovation, using a WS182b LED strip at the top of the steering wheel. LED lighting was controllable per track speed limits. Got good driver feedback!

- **Steering instrument cluster:**

Introduced first-of-its kind instrument cluster in the league. Drivers got live status updates like battery temp, SOC percentage, controller temps, motor temps, RPMs, Speed, error codes, etc.

- **Kart Aerodynamics:** Reach a top speed of **104 km/h** using just **5.5 kW**

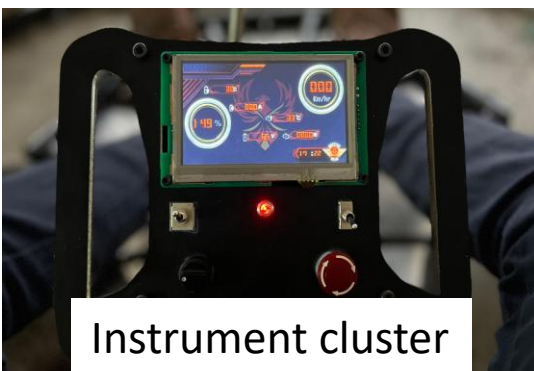
Reduced the **drag coefficient (Cd)** and **frontal area (Af)** of the kart using light weight full-body shell and aero elements in CAD, keeping airflow and packaging in mind.

Methodically optimized parameters like **Cd** (Co – Efficient of Drag), **μ_{rr}** (Co – Efficient of Rolling Resistance) and many others by testing & **SolidWorks Flow Simulation** to simulate different designs for aero elements. This helped us identify drag-heavy zones and improve the design.

Very proud that I taught myself aerodynamic concepts to contribute to optimally solve these problems

- **Light weighting:**

Using aerodynamics simulations, retrieved the wh/km, and used it in a battery pack simulation to optimize and build a **very light weight battery pack**.

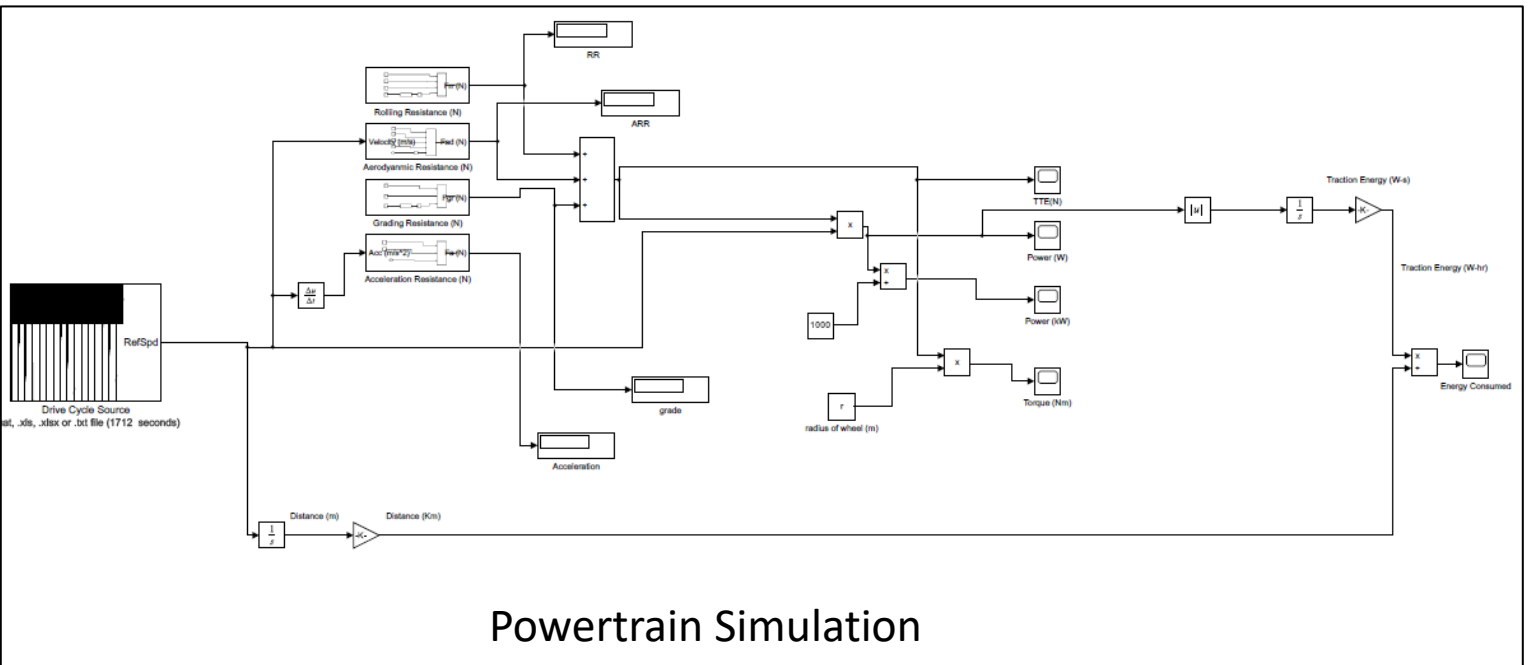
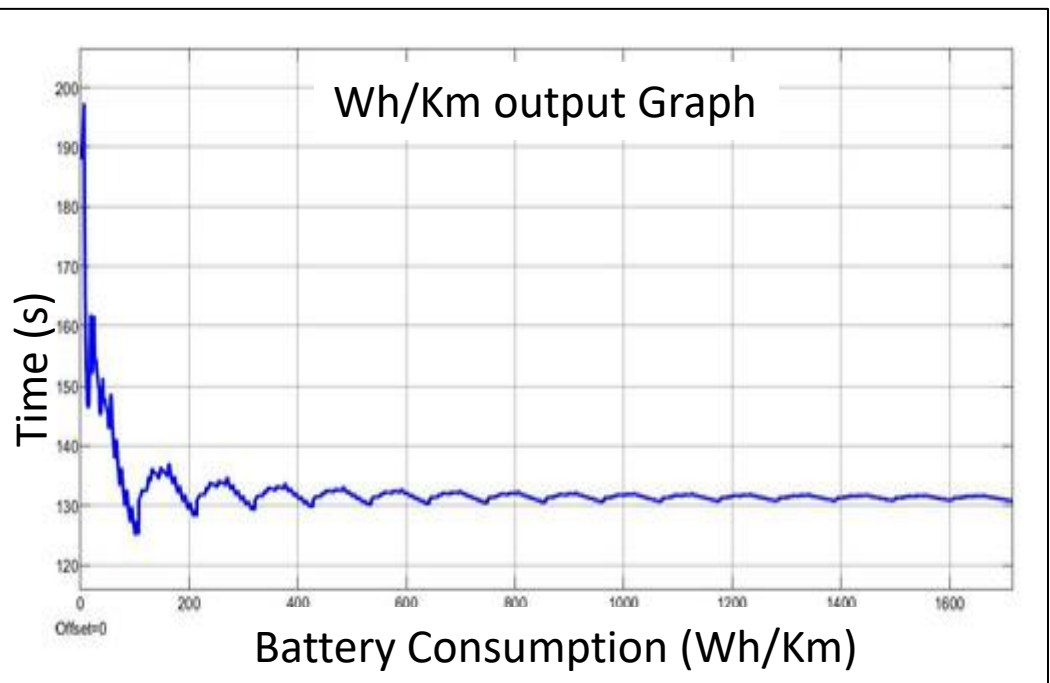
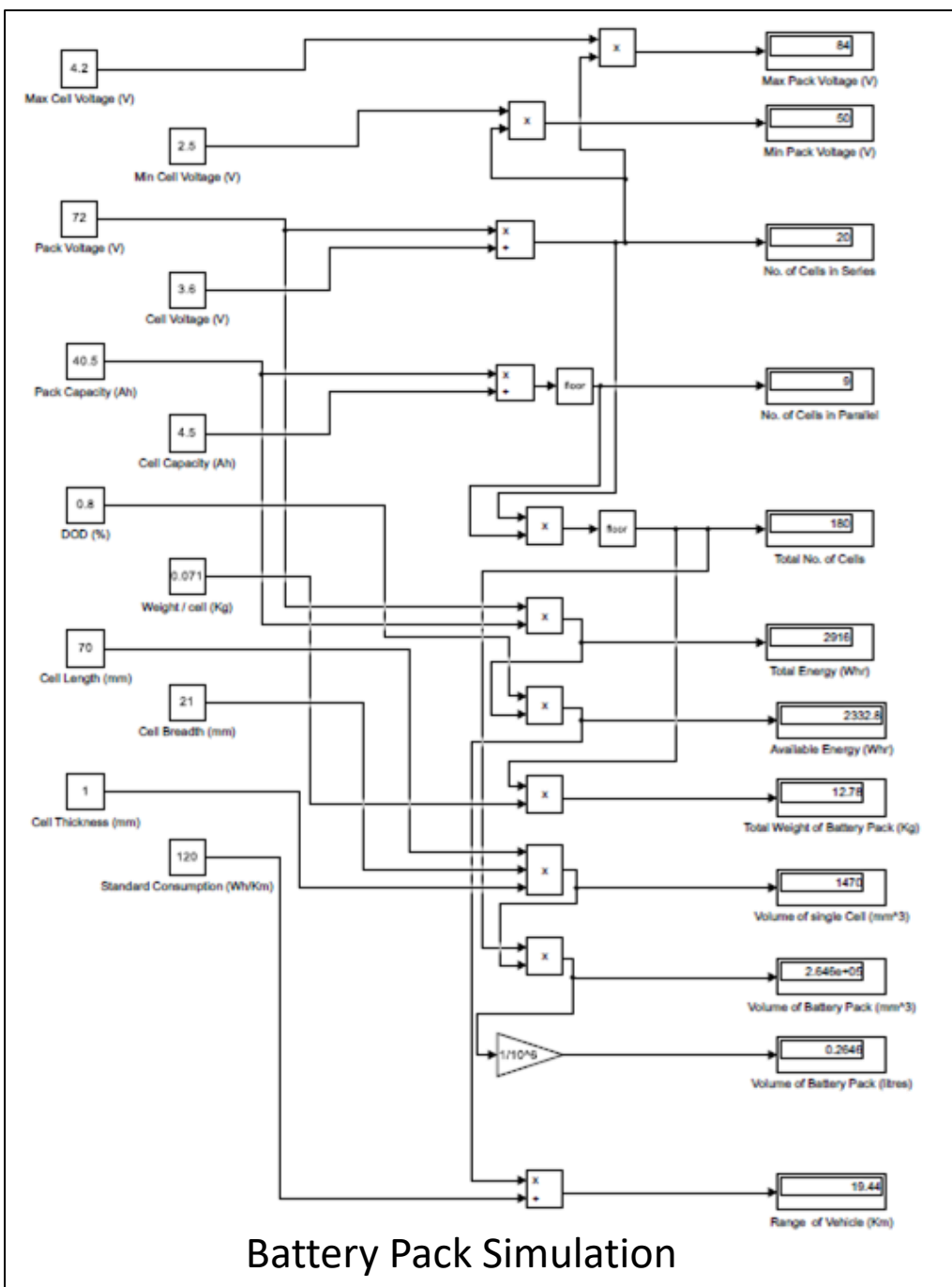
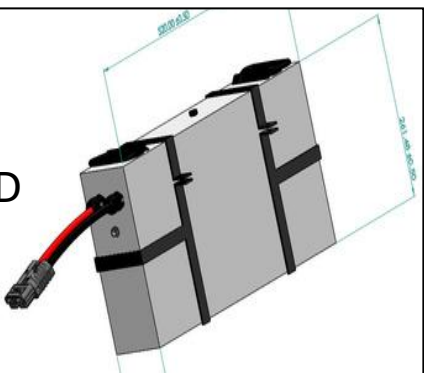


Instrument cluster



Light strip

Battery Pack CAD



GCAN – CAN Analyzer and Data Logger

Tech Stack

- **Hardware:** Raspberry Pi, STM32, MCP2515 CAN, Waveshare HDMI
- **Software:** Python, PySide6, SocketCAN, Linux
- **Protocols:** CAN Bus, SPI

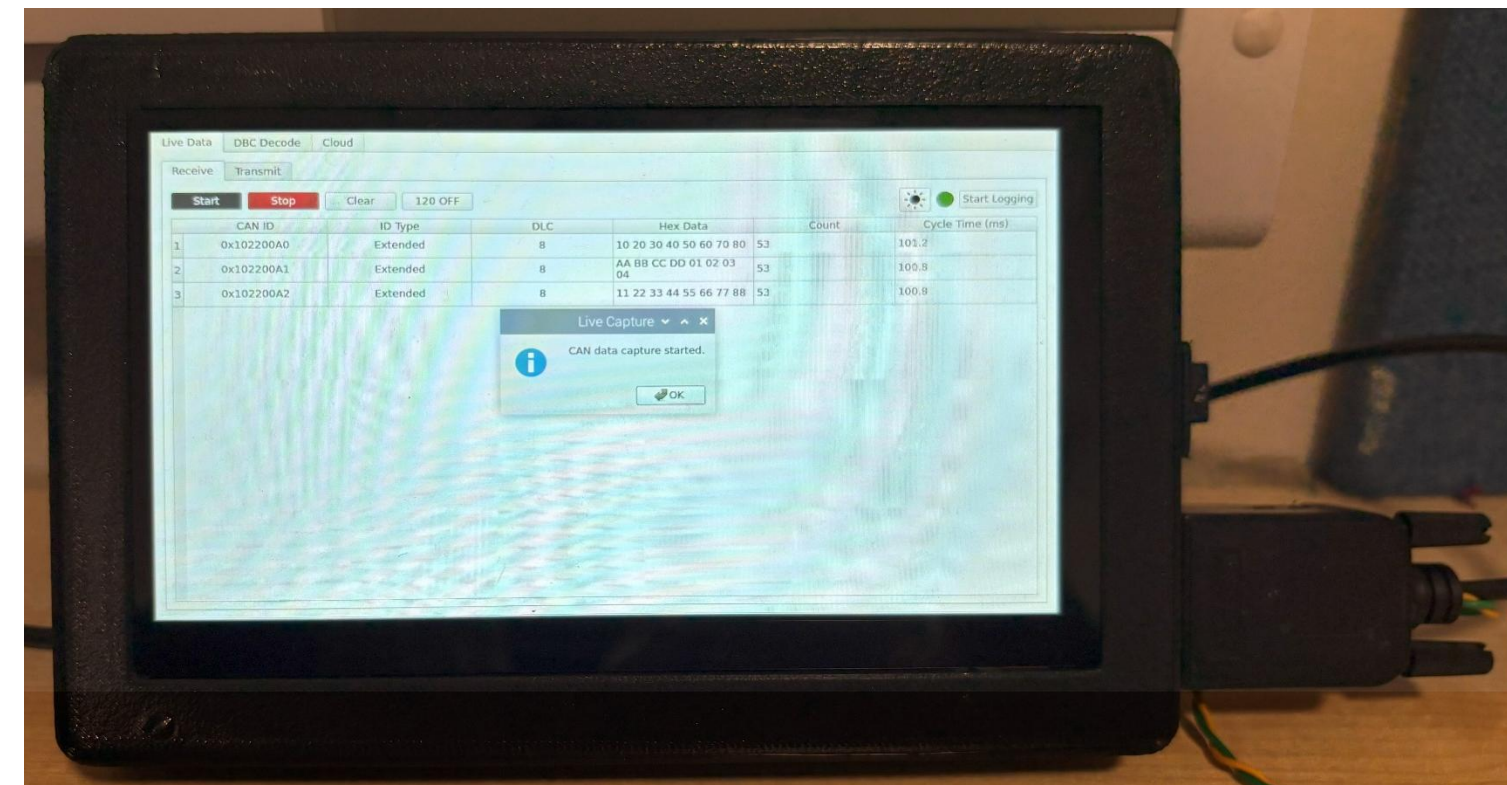
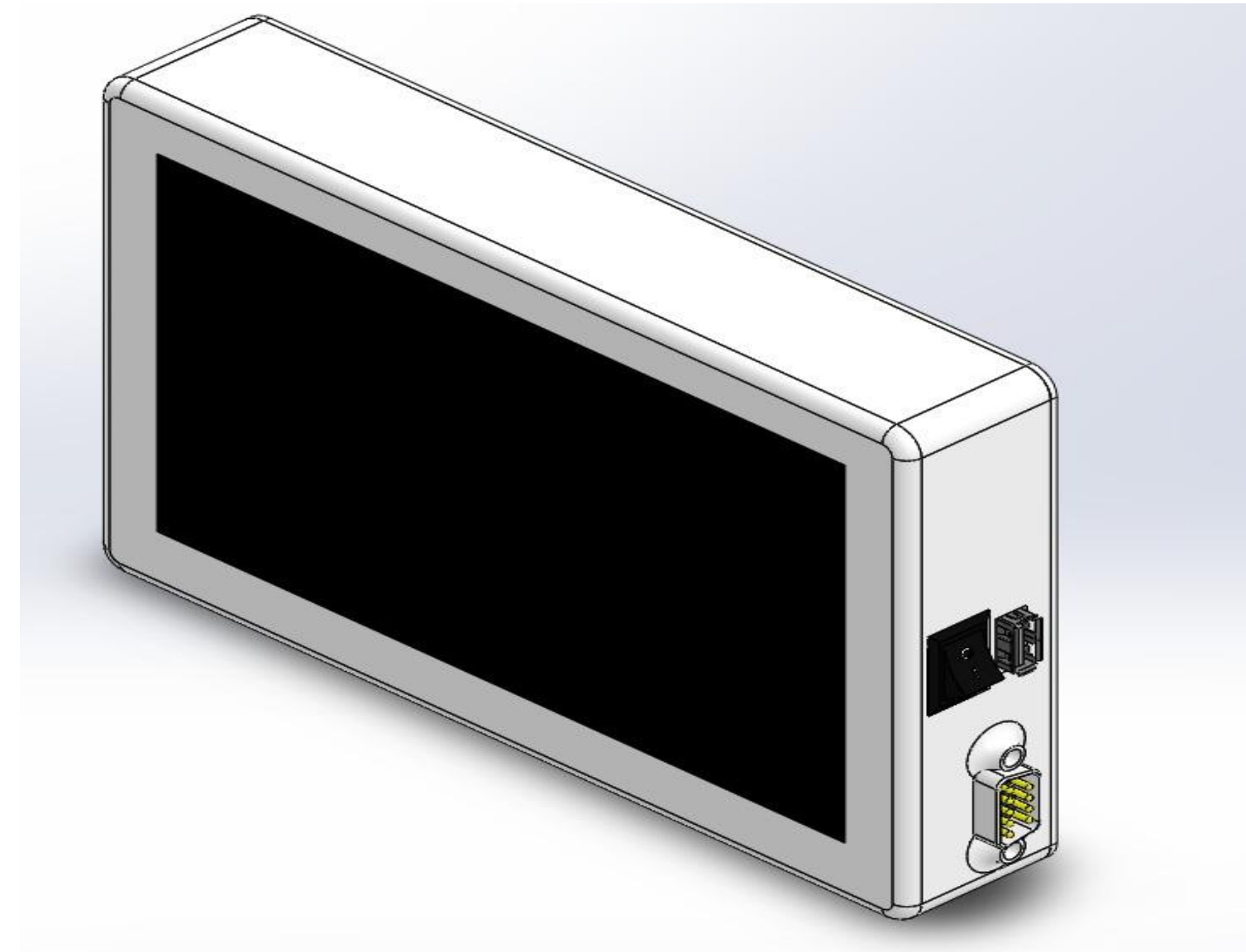
Overview

- Built a **real-time CAN bus data logger** using Raspberry Pi with a custom **Python GUI** for monitoring, decoding, and storing CAN data.

Highlights

- **GUI:** Multi-tab PySide6 interface with start/stop logging, error handling, and CSV export.
- **CAN Integration:** Configured MCP2515 + SocketCAN.
- **Data Handling:** Real-time logging to CSV/SQLite with tabular & list views.
- **Enhancements:** Display brightness control, modular design for future cloud/analytics features.

Link: [Project Page](#)



Drone Project

Link: [Project Page](#)

- HAWK v1.0
Problem statement: Geo-tagging of plantation in the catchment area of hydro project:
- As part of **Smart India Hackathon 2023**, our team built an autonomous drone system for the problem statement: *"Geo-tagging of plantation in the catchment area of hydro project"*, given by the Ministry of Power. The goal was to track plantation growth and detect early signs of plant stress or disease using aerial monitoring.
- The drone platform included a **Jetson Nano**, which ran a machine learning model in Microsoft Azure to detect crop disease in real time using data from **RGB and NIR cameras**. The system also collected **GPS-tagged images**, enabling us to map the plantation area and analyze vegetation health using **NDVI and EVI** indices.
- I was responsible for assembling the drone, tuning the **PID controllers** in Betaflight for stable flight, and programming the drone's firmware. I also worked on integrating the ML model with the Jetson Nano, and helped sync image data with the GPS module for accurate geo-tagging.
- After flight missions, we used **ArcGIS Pro** to process and visualize the data. Our solution was declared the winner at SIH 2023, and we were awarded a **₹1,00,000 cash prize** by the Ministry of Power for building a practical, field-ready system.



Website Development

- E-Kart Club Website

Built for our college's **Electric Kart Club** to showcase team updates, projects, and events

Collaborated with a teammate; focused on frontend development and GUI styling

Tech Stack Used:

- React
- Tailwind CSS
- Firebase (hosting & backend)

My Contributions:

- Wrote and styled the entire **frontend interface** using Tailwind CSS
- Designed the **GUI layout** and component structure
- Maintained and deployed updates to Firebase
- Ensured full responsiveness and performance optimization

- Garden Website – Freelance Project :

Designed for a **local garden services business** to create an online presence

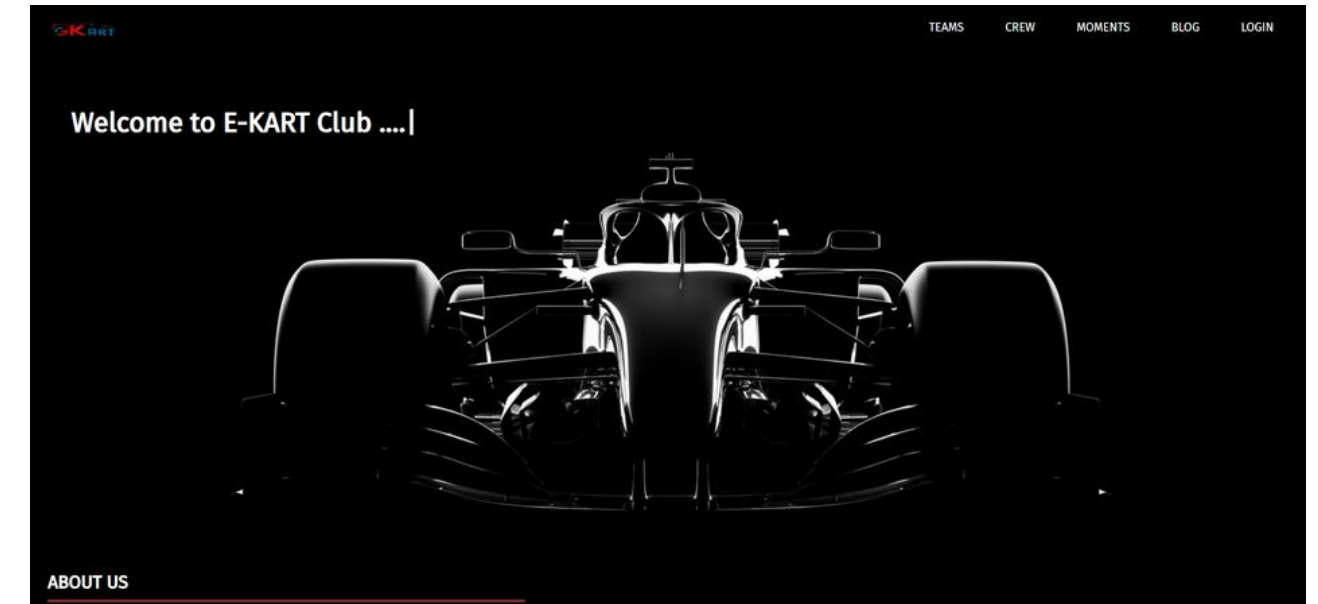
Focused on simplicity, fast loading, and mobile responsiveness

Tech Stack Used:

- HTML
- CSS
- Firebase (for hosting)

My Contributions:

- Developed the full static site from scratch
- Handled all **layout, styling, and deployment**
- Worked closely with the client to reflect their brand and services



Link : [Portfolio](#) | [E-Kart](#)



Link : [Garden](#)

The value I bring to the table

What I Bring to the Table

- Hands-on embedded ownership:** Comfortable with schematics, board bring-up, low-level C, and real hardware debugging.
- Reusable firmware thinking:** I build drivers, startup code, communication stacks, and tools that other engineers can extend.
- Systems integration:** I connect hardware, firmware, UI, logging, and testing so the full product works together.
- Measured problem solving:** I use profiling, simulation, and structured debugging to improve performance and reliability.
- Team value:** I document clearly, collaborate well, and stay useful across electrical, firmware, and product work.